

Wording for guaranteed copy elision through simplified value categories

ISO/IEC JTC1 SC22 WG21

P0135R1

Richard Smith

richard@metafoo.co.uk

2016-06-20

This paper provides wording for [P0135R0](#).

This also resolves the following core issues:

[1565](#): Copy elision and lifetime of `initializer_list` underlying array

[1590](#): Bypassing non-copy/move constructor copying

[1599](#): Lifetime of `initializer_list` underlying array

[1697](#): Lifetime extension and copy elision

[2022](#) [PARTIAL]: Copy elision in constant expressions (except for NRVO)

Change in 3.10 [basic.lval] paragraph 1:

Expressions are categorized according to the taxonomy in Figure 1. [Figure 1: Expression category taxonomy]

- An *lvalue* [...]
- [...]
- A *prvalue* [...]
- A *glvalue* is an expression whose evaluation determines the identity of an object, bit-field, or function.
- A *prvalue* is an expression whose evaluation initializes an object or a bit-field, or computes the value of the operand of an operator, as specified by the context in which it appears.
- An *xvalue* is a *glvalue* that denotes an object or bit-field whose resources can be reused (usually because it is near the end of its lifetime). [*Example*: Certain kinds of expressions involving *rvalue* references (8.3.2) yield *xvalues*, such as a call to a function whose return type is an *rvalue* reference or a cast to an *rvalue* reference type.]
- An *lvalue* is a *glvalue* that is not an *xvalue*.
- An *rvalue* is a *prvalue* or an *xvalue*.

[*Note*: Historically, *lvalues* and *rvalues* were so-called because they could appear on the left- and right-hand side of an assignment (although this is no longer generally true); *glvalues* are "generalized" *lvalues*, *prvalues* are "pure" *rvalues*, and *xvalues* are "eXpiring" *lvalues*. Despite their names, these terms classify expressions, not values.] Every expression belongs to exactly one of the fundamental classifications in this taxonomy: *lvalue*, *xvalue*, or *prvalue*. This property of an expression is called its *value category*. [...]

Add a new paragraph after 3.10 [basic.lval] paragraph 1:

The *result* of a *prvalue* is the value that the expression stores into its context. A *prvalue* whose result is the value *v* is sometimes said to have or name the value *v*. The *result object* of a *prvalue* is the object initialized by the *prvalue*; a *prvalue* that is used to compute the value of an operand of an operator or that has type (possibly cv-qualified) `void` has no result object. [*Note*: Except when the *prvalue* is the operand of a *decltype-specifier*, a *prvalue* of class or array type always has a result object. For a discarded *prvalue*, a temporary object is materialized; see [expr].] The *result* of a *glvalue* is the entity denoted by the expression.

Change in 3.10 [basic.lval] paragraph 2:

[*Note*: Whenever a *glvalue* appears in a context where a *prvalue* is expected, the *glvalue* is converted to a *prvalue*; see 4.1, 4.2, and 4.3.] [*Note*: ...] [*Note*: ...]

Add a new paragraph after 3.10 [basic.lval] paragraph 2:

[*Note*: Whenever a *prvalue* appears in a context where an *xvalue* or *glvalue* is expected, the *prvalue* is converted to an *xvalue*; see [conv.rval].]

Delete 3.10 [basic.lval] paragraphs 5 - 8 and change in paragraph 9:

~~If an expression can be used to modify the object to which it refers, the expression is called modifiable.~~ An *lvalue* is *modifiable* unless its type is `const`-qualified or is a function type. [*Note*: A program that attempts to modify an object through a nonmodifiable *lvalue* **expression** or **through an** *rvalue* expression is ill-formed ([expr.ass], [expr.post.inc], [expr.pre.inc]).]

Change in 4.1 [conv.lval] paragraph 1, second footnote (footnote 55):

In C++ class **and array** prvalues can have cv-qualified types ~~(because they are objects)~~. This differs from ISO C, in which non-lvalues never have cv-qualified types.

Change in 4.1 [conv.lval] paragraph 2:

When an lvalue-to-rvalue conversion is applied to an expression *e*, and [...] the value contained in the referenced object is not accessed. [*Example*: ...] ~~In all other cases,~~ The result of the conversion is determined according to the following rules:

- If *T* is (possibly cv-qualified) `std::nullptr_t`, the result is a null pointer constant (4.10).
- Otherwise, if *T* has a class type, the conversion copy-initializes ~~a temporary of type *T*~~ **the result object** from the glvalue ~~and the result of the conversion is a prvalue for the temporary.~~
- Otherwise, if the object to which the glvalue refers contains an invalid pointer value (3.7.4.2, 3.7.4.3), the behavior is implementation-defined.
- Otherwise, the value contained in the object indicated by the glvalue is the prvalue result.

Change in 4.2 [conv.array] paragraph 1:

An lvalue or rvalue of type "array of *N T*" or "array of unknown bound of *T*" can be converted to a prvalue of type "pointer to *T*". **The temporary materialization conversion ([conv.rval]) is applied.** The result is a pointer to the first element of the array.

Add a new subclause after 4.3:

4.4 Temporary materialization conversion [conv.rval]

A prvalue of type τ can be converted to an xvalue of type τ . This conversion initializes a temporary object ([class.temporal]) of type τ from the prvalue by evaluating the prvalue with the temporary object as its result object, and produces an xvalue denoting the temporary object. τ shall be a complete type. [*Note*: If τ is a class type (or array thereof), it must have an accessible and non-deleted destructor; see [class.dtor].] [*Example*:

```
struct X { int n; }  
int k = X().n; // ok, X() prvalue is converted to xvalue
```

1

Add a new paragraph after 5 [expr] paragraph 9 ("Whenever a glvalue expressions as an operand of an operator that expects a prvalue"):

Whenever an rvalue expression appears as an operand of an operator that expects a glvalue for that operand, the temporary materialization conversion ([conv.rval]) is applied to convert the expression to an xvalue.

Change in 5 [expr] paragraph 11:

In some contexts, an expression only appears for its side effects. Such an expression is called a *discarded-value expression*. ~~The expression is evaluated and its value is discarded.~~ The array-to-pointer (4.2) and function-to-pointer (4.3) standard conversions are not applied. The lvalue-to-rvalue conversion (4.1) is applied if and only if the expression is a glvalue of volatile-qualified type and it is one of the following: [...] [*Note*: Using an overloaded operator causes a function call; the above covers only operators with built-in meaning.] **If the expression is a prvalue after this optional conversion, the temporary materialization conversion ([temp.rval]) is applied.** [*Note*: If the **expression is an** lvalue **is** of class type, it must have a volatile copy constructor to initialize the temporary that is the result **object** of the lvalue-to-rvalue conversion.] **The glvalue expression is evaluated and its value is discarded.**

Drafting note: it is no longer meaningful to evaluate a prvalue expression of class or array type without specifying an object to initialize from it, as the prvalue does not itself create such an object.

Change in 5.1.2 [expr.prim.lambda] paragraph 2:

The evaluation of a *lambda-expression* results in a prvalue ~~temporary (12.2). This temporary is~~ **whose result object is** called the *closure object*. [...]

Change in 5.2.1 [expr.sub] paragraph 1:

A postfix expression followed by an expression in square brackets is a postfix expression. One of the expressions shall ~~have the~~ **be a glvalue** of type "array of T" or **a prvalue of type** "pointer to T" and the other shall ~~have be a~~ **prvalue of** unscoped enumeration or integral type. [...] in the case of an array operand, the result is an lvalue if that operand is an lvalue and an xvalue otherwise.

Change in 5.2.2 [expr.call] paragraph 4:

~~[...] During the initialization of a parameter, an implementation may avoid the construction of extra temporaries by combining the conversions on the associated argument and/or the construction of temporaries with the initialization of the parameter (see 12.2). The~~ **It is implementation-defined whether the** lifetime of a parameter ends when the function in which it is defined returns **or at the end of the enclosing full-expression**. [...] The ~~value~~ **result** of a function call is the ~~value returned by~~ **result of the operand of the evaluated return statement ((stmt.return)) in** the called function **(if any)**, except in a virtual function call if the return type of the final overrider is different from the return type of the statically chosen function, the value returned from the final overrider is converted to the return type of the statically chosen function.

Delete 5.2.2 [expr.call] paragraph 11:

~~If a function call is a prvalue of object type:~~

- ~~if the function call is either~~
 - ~~the operand of a decltype-specifier or~~
 - ~~the right operand of a comma operator that is the operand of a decltype-specifier;~~~~a temporary object is not introduced for the prvalue. The type of the prvalue may be incomplete. [Note: ...] [Note: ...]~~
- ~~otherwise, the type of the prvalue shall be complete.~~

Change in 5.2.3 [expr.type.conv]:

1. A *simple-type-specifier* (7.1.6.2) or *typename-specifier* (14.6) followed by a parenthesized **optional** *expression-list* **or by a braced-init-list (the initializer)** constructs a value of the specified type given the ~~expression list~~ **initializer**. If the ~~expression list~~ **initializer** is a **parenthesized** single expression, the type conversion expression is equivalent (in definedness, and if defined in meaning) to the corresponding cast expression (5.4). **If the type is (possibly cv-qualified) void and the initializer is (), the expression is a prvalue of the specified type that performs no initialization. Otherwise, the expression is a prvalue of the specified type whose result object is direct-initialized (8.5) with the initializer. For an expression of the form $\tau()$, τ shall not be an array type. [...]**

2. [...]

3. [...]

Change in 5.2.5 [expr.ref] paragraph 2:

For the first option (dot) the first expression shall **be a glvalue having** ~~have~~ complete class type. [...]

Drafting note: if the left operand is a prvalue, a temporary will be materialized and it will be converted to an xvalue per [expr]/9+.

Change in 5.2.7 [expr.dynamic.cast] paragraph 2:

[...] If τ is an rvalue reference type, v shall be ~~an expression~~ **a glvalue** having a complete class type, and the result is an xvalue of the type referred to by τ .

Drafting note: a temporary is materialized if the operand is a prvalue.

Change in 5.2.9 [expr.static.cast] paragraph 3:

An ~~glvalue, class prvalue, or array prvalue~~ of type "cv1 T1" can be cast to type "rvalue reference to cv2 T2" if "cv2 T2" is reference-compatible with "cv1 T1" (8.5.3). If the value is not a bit-field, the result refers to the object or the specified base class subobject thereof; otherwise, the lvalue-to-rvalue conversion (4.1) is applied to the bit-field and the resulting prvalue is used as the expression of the static_cast for the remainder of this section. If T2 is an inaccessible (Clause 11) or ambiguous (10.2) base class of T1, a program that necessitates such a cast is ill-formed.

Drafting note: the deleted cases would also be handled by the immediately-following paragraph; we only need the special case above to convert lvalues to xvalues.

Change in 5.2.9 [expr.static.cast] paragraph 4:

An expression e can be explicitly converted to a type τ using a `static_cast` of the form `static_cast< $\tau$ >(e)` if the declaration $\tau \ t(e);$ is well-formed, for some invented temporary variable t (8.5). **The effect of such an explicit conversion** **If τ is a reference type, the effect** is the same as performing the declaration and initialization and then using the temporary variable as the result of the conversion. **Otherwise, the result object is direct-initialized from e .** ~~The expression e is used as a glvalue if and only if the initialization uses it as a glvalue.~~

Change in 5.2.11 [expr.const.cast] paragraph 4:

[...]
The result of a reference `const_cast` refers to the original object **if the operand is a glvalue and to the result of applying the temporary materialization conversion ([conv.rval]) otherwise.**

Change in 5.2.8 [expr.typeid] paragraph 3:

When `typeid` is applied to an expression other than a glvalue of a polymorphic class type, the result refers to a `std::type_info` object representing the static type of the expression. Lvalue-to-rvalue (4.1), array-to-pointer (4.2), and function-to-pointer (4.3) conversions are not applied to the expression. ~~If the type of the expression is a class type, the class shall be completely defined.~~ **If the expression is a prvalue, the temporary materialization conversion ([conv.rval]) is applied.** The expression is an unevaluated operand (Clause 5).

Change in 5.3.3 [expr.sizeof] paragraph 4:

The lvalue-to-rvalue (4.1), array-to-pointer (4.2), and function-to-pointer (4.3) standard conversions are not applied to the operand of `sizeof`. **If the operand is a prvalue, the temporary materialization conversion ([conv.rval]) is applied.**

Drafting note: this maintains the status quo. We could alternatively remove these conversions and allow `typeid` and `sizeof` to behave more like `decltype`.

Change in 5.5 [expr.mptr.oper] paragraph 2:

The binary operator `.*` binds its second operand, which shall be **a glvalue** of type "pointer to member of τ " to its first operand, which shall be of class τ or of a class of which τ is an unambiguous and accessible base class. The result is an object or a function of the type specified by the second operand.

Change in 5.16 [expr.cond] bullet 7.1:

The second and third operands have the same type; the result is of that type **and the result object is initialized using the selected operand.** ~~If the operands have class type, the result is a prvalue temporary of the result type, which is copy-initialized from either the second operand or the third operand depending on the value of the first operand.~~

Change in 5.19 [expr.comma] paragraph 1:

[...] If the value of the right operand is a temporary **expression** (12.2), the result is that temporary **expression**.

Change in 6.6.3 [stmt.return] paragraph 2:

[...] A return statement with any other operand shall be used only in a function whose return type is not `cv void`; the return statement initializes the ~~object or reference to be returned~~ **glvalue result or prvalue result object of the (explicit or implicit) function call** by copy-initialization (8.5) from the operand. [*Note:* A return statement can involve ~~the construction and copy or move of a temporary object (12.2);~~ **an invocation of a constructor to perform a copy or move of the operand if it is not a prvalue or if its type differs from the return type of the function.** A copy ~~or move~~ operation associated with a return statement may be elided or ~~considered as an rvalue for the purpose of overload resolution in selecting a constructor~~ **converted to a move operation if an automatic storage duration variable is returned** (12.8).] [...]

Drafting note: while we no longer create elidable temporaries, we still have optional elision for the NRVO cases, and still convert copies to moves in some cases.

Change in 6.6.3 [stmt.return] paragraph 3:

The copy-initialization of the ~~returned entity~~ **result of the call** is sequenced before [...]

Change in 7.1.6.2 [dcl.type.simple] paragraph 5:

If the operand of a *decltype-specifier* is a prvalue, the temporary materialization conversion is not applied ([*conv.rval*]) and no result object is provided for the prvalue. The type of the prvalue may be incomplete.

[*Note*: in the case where the operand of a *decltype-specifier* is a function call and the return type of the function is a class type, a special rule (5.2.2) ensures that the return type is not required to be complete (as it would be if the call appeared in a sub-expression or outside of a *decltype-specifier*). As a result, storage is not allocated for the prvalue and it is not destroyed. Thus, a class type is not instantiated as a result of being the type of a function call in this context. In this context, the common purpose of writing the expression is merely to refer to its type. In that sense, a *decltype-specifier* is analogous to a use of a *typedef-name*, so the usual reasons for requiring a complete type do not apply. In particular, it is not necessary to allocate storage for a temporary object or to enforce the semantic constraints associated with invoking the type's destructor.] [*Note*: Unlike the preceding rule, parentheses have no special meaning in this context.] [*Example*: ...]

Change in 8.5 [dcl.init] bullet 17.6:

If the destination type is a (possibly cv-qualified) class type):

- If the initializer expression is a prvalue and the cv-unqualified version of the source type is the same class as the class of the destination, the initializer expression is used to initialize the destination object. [*Example*: `T x = T(T());`; calls the `T` default constructor to initialize `x`.]
- Otherwise, if the initialization is direct-initialization, or if it is copy-initialization where the cv-unqualified version of the source type is the same class as, or a derived class of, the class of the destination, constructors are considered. [...]
- Otherwise (i.e., for the remaining copy-initialization cases), user-defined conversion sequences that can convert from the source type to the destination type or (when a conversion function is used) to a derived class thereof are enumerated as described in 13.3.1.4, and the best one is chosen through overload resolution (13.3). If the conversion cannot be done or is ambiguous, the initialization is ill-formed. The function selected is called with the initializer expression as its argument; if the function is a constructor, the call is a prvalue initializes a temporary of the cv-unqualified version of the destination type whose return object is initialized by the constructor. The temporary is a prvalue. The result of the call (which is the temporary for the constructor case) is then used to direct-initialize, according to the rules above, the object that is the destination of the copy-initialization. In certain cases, an implementation is permitted to eliminate the copying inherent in this direct-initialization by constructing the intermediate result directly into the object being initialized; see 12.2, 12.8.

Change in 8.5.3 [decl.init.ref] bullet 5.2.1:

If the initializer expression

- is an xvalue rvalue (but not a bit-field), class prvalue, array prvalue or function lvalue and "cv1 T1" is reference-compatible with "cv2 T2", or
- has a class type (i.e., T2 is a class type), where T1 is not reference-related to T2, and can be converted to an xvalue, class prvalue, rvalue or function lvalue of type "cv3 T3", where "cv1 T1" is reference-compatible with "cv3 T3" (see 13.3.1.6),

then the reference is bound to the value of the initializer expression in the first case and to the result of the conversion in the second case (or, in either case, to an appropriate base class subobject) after applying the temporary materialization conversion ([*conv.rval*]). [*Example*: ...]

Drafting note: this means that some cases now bind directly that did not before:

```
struct X {};  
X &&a = X(); // binds directly  
int &&b = int(); // used to not bind directly, now does
```

This does not appear to affect any of the places where we use the term "bind directly".

Change in 8.5.3 [decl.init.ref] bullet 5.2.2:

Otherwise

- If T1 or T2 is a class type and T1 is not reference-related to T2, user-defined conversions are considered using the rules for copy-initialization of an object of type "cv1 T1" by user-defined conversion (8.5, 13.3.1.4, 13.3.1.5); the program is ill-formed if the corresponding non-reference copy-initialization would be ill-formed. The result of the call to the conversion function, as described for the non-reference copy-initialization, is then used to direct-initialize the reference. For this direct-initialization, user-defined conversions are not considered.

- Otherwise, a temporary of type "cv1 T1" is created and copy-initialized (8.5) from the initializer expression. The reference is then bound to the temporary: **the initializer expression is implicitly converted to a prvalue of type "cv T1". The temporary materialization conversion is applied and the reference is bound to the result.**

If T1 is reference-related to T2:

- cv1 shall be the same cv-qualification as, or greater cv-qualification than, cv2; and
- if the reference is an rvalue reference, the initializer expression shall not be an lvalue.

[*Example: ...*] In all cases except the last (i.e., ~~creating and initializing a temporary from the initializer expression~~ **implicitly converting the initializer expression to the underlying type of the reference**), the reference is said to *bind directly* to the initializer expression.

Change in 8.5.4 [dcl.init.list] bullet 3.5:

Otherwise, if τ is a specialization of `std::initializer_list<E>`, ~~a prvalue `initializer_list`~~ **the** object is constructed as described below ~~and used to initialize the object according to the rules for initialization of an object from a class of the same type (8.5).~~

Change in 8.5.4 [dcl.init.list] bullet 3.8:

Otherwise, if τ is a reference type, a prvalue ~~temporary~~ of the type referenced by τ is **generated. The prvalue initializes its result object by** copy-list-initialized ~~ation~~ or direct-list-initialized ~~ation~~, depending on the kind of initialization for the reference; ~~and the reference is bound to that temporary.~~ **The prvalue is then used to direct-initialize the reference.** [*Note:* As usual, the binding will fail and the program is ill-formed if the reference type is an lvalue reference to a non-const type.]

Change in 8.5.4 [dcl.init.list] paragraph 5:

An object of type `std::initializer_list<E>` is constructed from an initializer list as if the implementation ~~allocated a temporary~~ **generated and materialized ([conv.rval]) a prvalue of type** "array of N ~~elements of type~~ `const E`", where N is the number of elements in the initializer list. [...]

Change in 12.2 [class.temporary]:

1. Temporaries of class type Temporary objects are created ~~in various contexts:~~ **when a prvalue is materialized so that it can be used as a glvalue ([conv.rval])** [*Footnote:* This occurs

- when** binding a reference to a prvalue (8.5.3, 5.2.3, 5.2.7, 5.2.9, 5.2.11, 5.4),
- when initializing an object of type `std::initializer_list<T>` from a *braced-init-list* (8.5.4),**
- when performing member access on a class prvalue (5.2.5, 5.5),**
- when performing an array-to-pointer conversion or subscripting on an array prvalue (5.2.1),**
- for certain unevaluated operands (5.2.8, 5.3.3), and**
- when a prvalue appears as a discarded-value expression (5).**

] returning a prvalue (6.6.3), a conversion that creates a prvalue (4.1, 5.2.9, 5.2.11, 5.4), when needed by the implementation to pass or return an object of trivially-copyable type (see below), and when throwing an exception (15.1), ~~and in some initializations (8.5).~~ [*Note:* The lifetime of exception objects is described in 15.1.] Even when the creation of the temporary object is unevaluated (Clause 5) ~~or otherwise avoided (12.8)~~, all the semantic restrictions shall be respected as if the temporary object had been created and later destroyed. [*Note:* This includes accessibility (11) and whether it is deleted, for the constructor selected and for the destructor. However, in the special case of ~~a function call used as~~ the operand of a *decltype-specifier* (5.2.2), no temporary is introduced, so the foregoing does not apply to the prvalue of any such function call.]

2. The materialization of a temporary object is generally delayed as long as possible in order to avoid creating unnecessary temporary objects. [*Example:* Consider the following code:

```
class X {
public:
    X(int);
    X(const X&);
    X& operator=(const X&); ~X();
};
class Y {
public:
```

```

    Y(int); Y(Y&&); ~Y();
};
X f(X);
Y g(Y);
void h() {
    X a(1);
    X b = f(X(2));
    Y c = g(Y(3));
    a = f(a);
}

```

An implementation might use a temporary in which to construct X(2) before passing it to f() using Xâ€™s copy constructor; alternatively, X(2) might be is constructed in the space used to hold the f()'s argument. Likewise, an implementation might use a temporary in which to construct Y(3) before passing it to g() using Yâ€™s move constructor; alternatively, and Y(3) might be is constructed in the space used to hold the g()'s argument. Also, a temporary might be used to hold the result of f(X(2)) before copying it to b using Xâ€™s copy constructor; alternatively, Likewise, f()'s result might be is constructed directly in b. Likewise, a temporary might be used to hold the result of g(Y(3)) before moving it to c using Yâ€™s move constructor; alternatively, and g()'s result might be is constructed directly in c. On the other hand, the expression a = f(a) requires a temporary for the result of f(a), which is then assigned to a materialized so that the reference parameter of A::operator=(const A&) can bind to it .]

2+. When an object of class type x is passed to or returned from a function, if each copy constructor, move constructor, and destructor of x is either trivial or deleted, and x has at least one non-deleted copy or move constructor, implementations are permitted to create a temporary object to hold the function parameter or result object. The temporary object is constructed from the function argument or return value, respectively, and the function's parameter or return object is initialized as if by using the non-deleted trivial constructor to copy the temporary (even if that constructor is inaccessible or would not be selected by overload resolution to perform a copy or move of the object). [Note: This latitude is granted to allow objects of class type to be passed to or returned from functions in registers.]

3. [...]

Change in 12.4 [class.dtor] paragraph 11 bullet 4:

- for a constructed temporary object when its lifetime ends ([conv.rval], [class.temporary])

Change in 12.8 [class.copy] bullet 31.1:

[...] the copy/move operation can be omitted by constructing the automatic object directly into the function call's return value object

Delete 12.8 [class.copy] bullet 31.3:

when a temporary class object that has not been bound to a reference (12.2) would be copied/moved to a class object with the same type (ignoring cv-qualification), the copy/move operation can be omitted by constructing the temporary object directly into the target of the omitted copy/move

Change in 12.8 [class.copy] paragraph 31's example:

```

class Thing {
public:
    Thing();
    ~Thing();
    Thing(const Thing&);
};

Thing f() {
    Thing t;
    return t;
}

Thing t2 = f();

```

Here the criteria for elision can be combined to eliminate two calls to the copy constructor of class Thing: the copying of the local automatic object t into the temporary result object for the return value of function call f() and the copying of that temporary object into object t2 which is the global object t2. Effectively, the construction of the

local object t can be viewed as directly initializing the global object t_2 , and that object's destruction will occur at program exit. Adding a move constructor to `Thing` has the same effect, but it is the move construction from the ~~temporary~~ **local automatic** object to t_2 that is elided.

Change in 13.3.1 [over.match.funcs] paragraph 5:

During overload resolution, the implied object argument is indistinguishable from other arguments. The implicit object parameter, however, retains its identity since ~~conversions on the corresponding argument shall obey these additional rules:~~

- ~~no temporary object can be introduced to hold the argument for the implicit object parameter;~~ and
- no user-defined conversions can be applied to achieve a type match with it.

[...]

Change in 13.3.3.1 [over.best.ics] paragraph 7:

In all contexts, when converting to the implicit object parameter or when converting to the left operand of an assignment operation only standard conversion sequences ~~that create no temporary object for the result~~ are allowed.

Drafting note: standard conversion sequences never create temporary objects, so this wording appears to be redundant before this change, and after this change it would prevent cases like $x().f()$, which creates a temporary object while binding the implicit object parameter to the $x()$ prvalue.

Change in 13.3.3.1.4 [over.ics.ref] paragraph 3:

[...] [*Note*: This means, for example, that a candidate function cannot be a viable function if it has a non-const lvalue reference parameter (other than the implicit object parameter) and the corresponding argument ~~is a temporary or~~ would require ~~one~~ **a temporary** to be created to initialize the lvalue reference (see 8.5.3).]

Change in 15.1 [except.throw] paragraph 3:

Throwing an exception copy-initializes (8.5, 12.8) a temporary object, called the *exception object*. ~~The temporary is an~~ **An** lvalue **denoting the temporary** ~~and~~ is used to initialize the variable declared in the matching handler (15.3).
[...]

Change in table 19 [defaultconstructible]:

an ~~temporary~~ object of type T is value-initialized or aggregate-initialized